Základy počítačové grafiky
Přednáška 4

Martin Němec

VŠB-TU Ostrava

2026

VŠB TECHNICKÁ UNIVERZITA OSTRAVA | FAKULTA ELEKTROTECHNIKY A INFORMATIKY | KATEDRA INFORMATIKY

Aktuálně

- Základní vykreslování objektů v OpenGL (VAO+VBO).
- Vytváření a kompilace základních shaderů.
- Transformace modelů ve 3D.
- Měli byste mít funkční základní vykreslování.
- Zdrojový kód by měl být vhodně objektově upraven.

glGetUniformLocation na https://khronos.org/

glGetUniformLocation Returns the location of a uniform variable
GLint glGetUniformLocation(GLuint program, GLchar *name);

program - Specifies the program object to be queried.
name - Points to a null terminated string containing the name of the uniform variable whose location is to be queried.
Description glGetUniformLocation returns an integer that represents the location of a specific uniform ...
Errors

- GL_INVALID_VALUE is generated if program is not a value generated by OpenGL.
- GL_INVALID_OPERATION is generated if program is not a program object.
- GL_INVALID_OPERATION is generated if program has not been successfully linked.

Kulová plocha

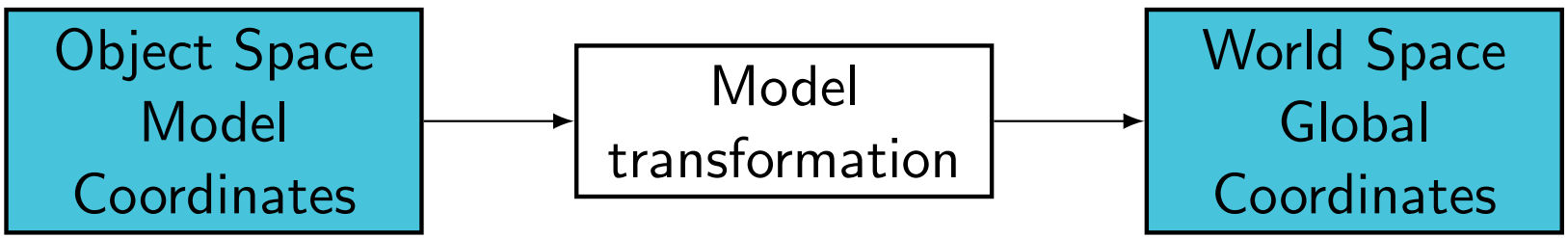
```
const float sphere[17280] = {
-0.831,-0.555,0.000,-0.833,-0.552,0.000,
-0.923,-0.382,0.000,-0.924,-0.380,0.000,
-0.815,-0.555,-0.162,-0.817,-0.552,-0.162,
...
-0.375,-0.923,0.074,-0.380,-0.921,0.075,
-0.555,-0.831,0.000,-0.559,-0.828,0.000};
```




```
glVertexAttribPointer(
  0,3,GL_FLOAT,GL_FALSE,6*sizeof(float),NULL);
glVertexAttribPointer(
  1,3,GL_FLOAT,GL_FALSE,6*sizeof(float),
     (GLvoid*)(3*sizeof(GLfloat)));
//How to convert other data types to float
//GL_TRUE - GL_BYTE <-128,127> -> <-1.0,1.0>
```

Opakování

- Převod z lokálního do globálního souřadného systému.
- Uniformní proměnná mat4 modelMatrix v GLSL.
- Skládání matic (komutativita).



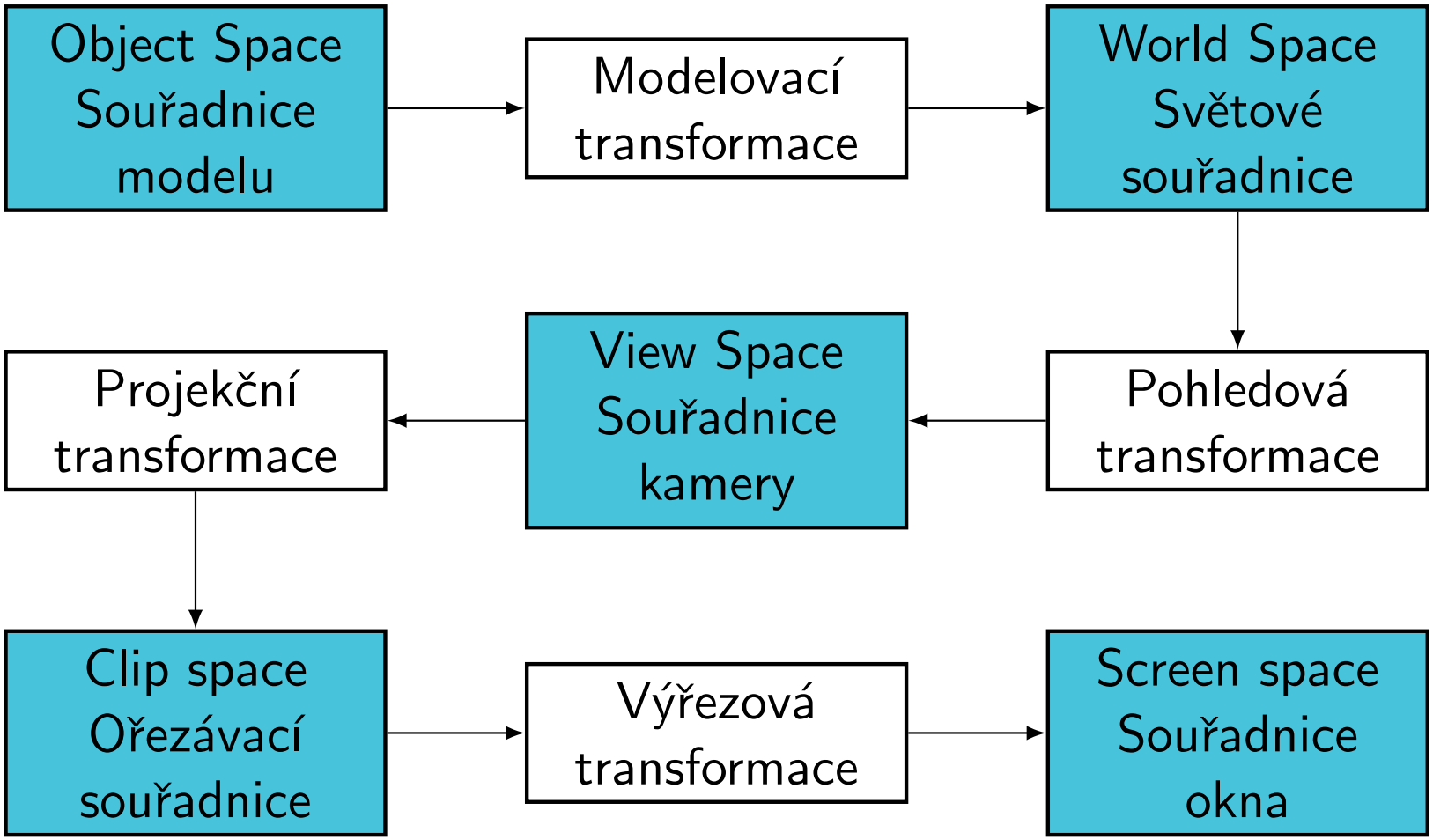
V čem je rozdíl (každý frame rotace o úhel alfa)?

```
alfa+=0.5;
M = rotate(mat4(1.0f),alfa,vec3(0.0f, 1.0f, 0.0f));
```

vs.

```
M = rotate(M, 0.5f, vec3(0.0f, 1.0f, 0.0f));
```

Aktuálně


Object Space
Souřadnice modelu
Modelovací transformace
World Space
Světové souřadnice
Pohledová transformace
View Space
Souřadnice kamery
Projekční transformace
Clip space
Ořezávací souřadnice
Výřezová transformace
Screen space
Souřadnice okna

Rotace bodu pomocí bázových vektorů

Rotaci bodu lze popsat pomocí **bázových vektorů**, které určují směry nových os vzhledem k původní souřadné soustavě.

Původní kartézská báze je tvořena jednotkovými vektory:

$$\hat{u}_1 = (1, 0, 0), \quad \hat{u}_2 = (0, 1, 0), \quad \hat{u}_3 = (0, 0, 1).$$

Zvolíme-li tři **navzájem kolmé jednotkové vektory** $\hat{a}, \hat{b}, \hat{c}$ (ortonormální báze), můžeme z nich sestavit rotační matici:

$$R = \begin{bmatrix} a_x & b_x & c_x \\ a_y & b_y & c_y \\ a_z & b_z & c_z \end{bmatrix} = \begin{bmatrix} | & | & | \\ \hat{a} & \hat{b} & \hat{c} \\ | & | & | \end{bmatrix}.$$

Sloupce matice tvoří nové bázové vektory vyjádřené v původní soustavě. **Rotaci bodu** A pak provedeme násobením jeho polohového vektoru maticí R jako $A' = \mathbf{R}A$.

Rotace pomocí bázových vektorů

Sestrojte matici rotace souřadné soustavy o 90° kolem osy z pomocí nových bázových vektorů.

Nová lokální souřadná soustava má následující orientaci:

- osa x' míří ve směru původní osy y,
- osa y' míří proti směru původní osy x,
- osa z' má stejný směr jako původní osa z.

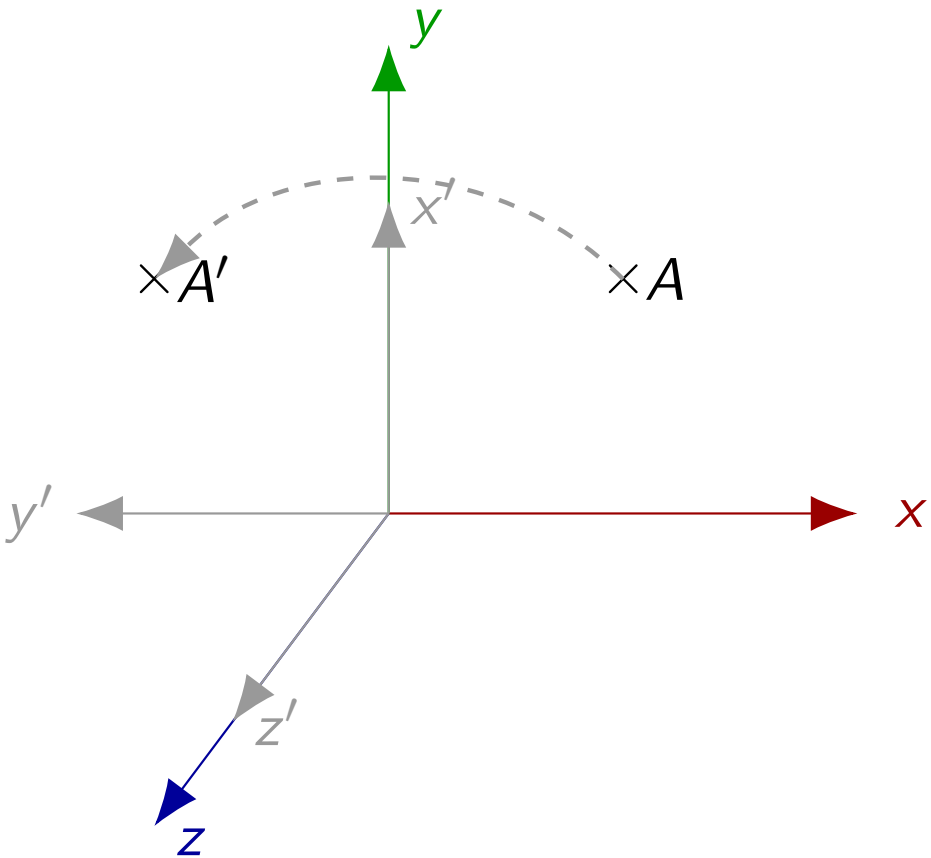
Nové bázové vektory vyjádřené v původní soustavě jsou:

$$\hat{u}_1 = (1, 0, 0), \quad \hat{u}_2 = (0, 1, 0), \quad \hat{u}_3 = (0, 0, 1)$$

Sestavením těchto vektorů do sloupců získáme rotační matici:

$$R = \begin{bmatrix} a_x & b_x & c_x \\ a_y & b_y & c_y \\ a_z & b_z & c_z \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Pohledová transformace



Rotace bodu pomocí rotační matice

Při rotaci bodu $A = [1, 1, 0]$ o úhel $90°$ kolem osy z pomocí rotační matice, vzniklé z bázových vektorů, dostaneme:

$$\mathbf{A}' = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \\ 0 \end{bmatrix}.$$

Výsledný bod má souřadnice $A' = [-1, 1, 0]$. Bod se otočil proti směru hodinových ručiček při pohledu ve směru kladné osy z.

Vyjádření souřadnic v nové bázi

Pokud chceme vyjádřit souřadnice pevného bodu A v nové, otočené bázi (nezaměňovat s rotací bodu), bod zůstává na svém místě a mění se pouze jeho souřadnicový popis.

Použijeme inverzní rotační matici. Protože rotační matice je ortogonální ($A^T A = I$), platí $R^{-1} = R^T$:

$$A' = \mathbf{R^T} A.$$

$$R = \begin{bmatrix} | & | & | \\ \hat{a} & \hat{b} & \hat{c} \\ | & | & | \end{bmatrix}, \qquad R^T = \begin{bmatrix} — & \hat{a}^T & — \\ — & \hat{b}^T & — \\ — & \hat{c}^T & — \end{bmatrix}.$$

Vyjádření souřadnic v nové bázi

Uvažujme bod $A = (1, 1, 0)$ a otočení soustavy o $90°$ kolem osy z. Transponovaná matice je:

$$R^T = \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \hat{a}^T \\ \hat{b}^T \\ \hat{c}^T \end{bmatrix}.$$

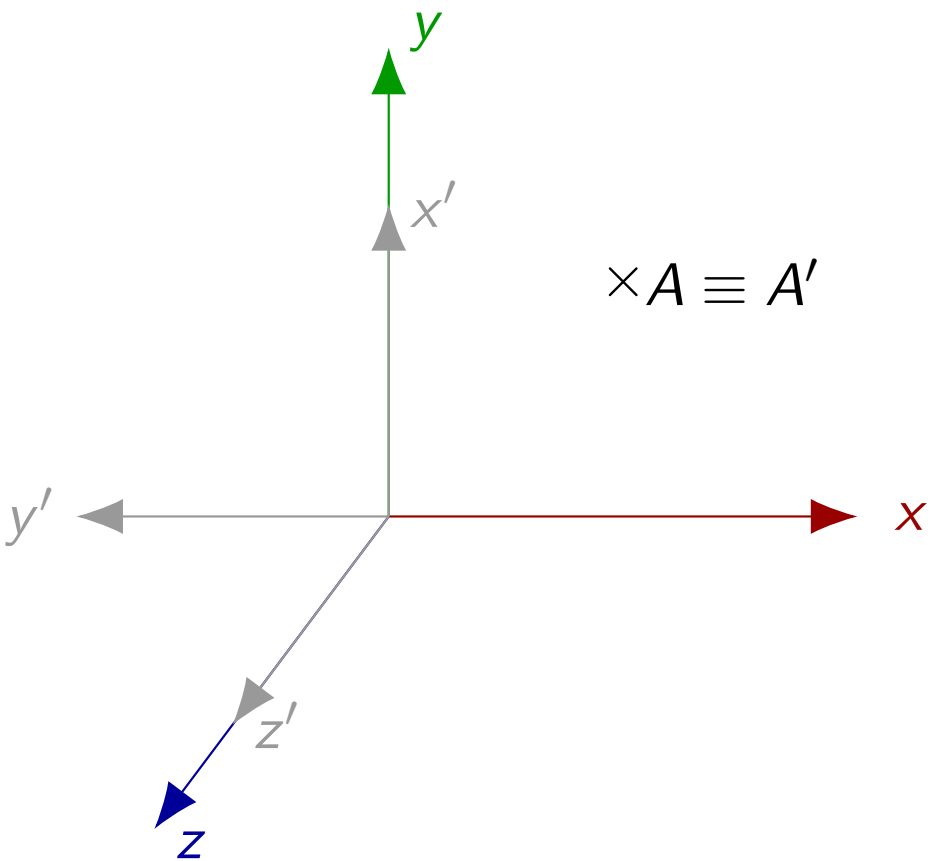
Souřadnice bodu A' v nové bázi získáme výpočtem:

$$A' = \mathbf{R^T} A$$

$$\begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix}.$$

Výsledek: Bod A má v nové bázi souřadnice $A' = [1, -1, 0]$.

Vyjádření souřadnic v nové bázi



Pohledová transformace

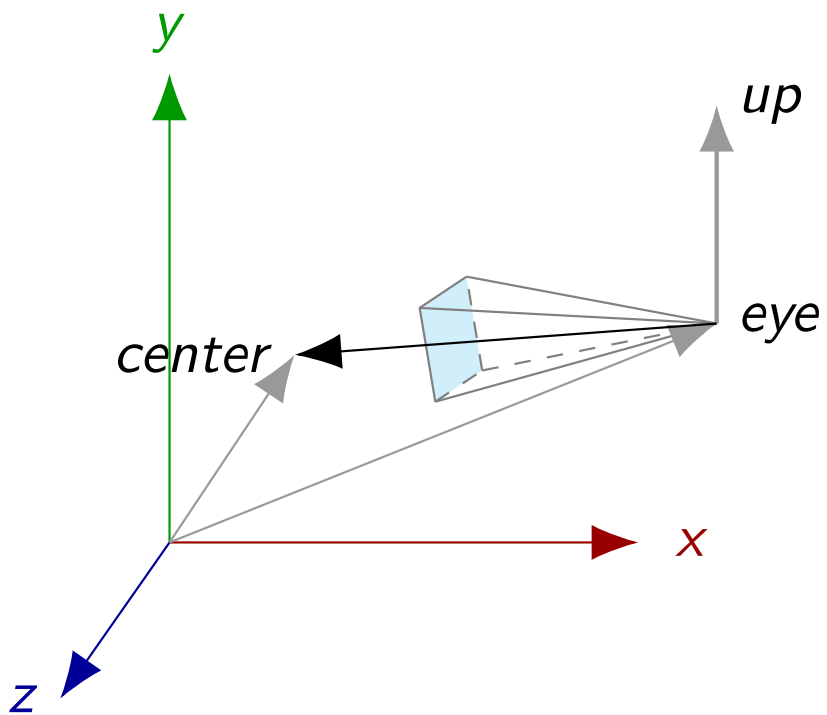
Transformace objektů nám umožňují měnit jejich polohu ve scéně. Potřebujeme také určit, **odkud a kam se díváme**.

- Ve výchozím nastavení se kamera nachází v počátku souřadnic, dívá se podél záporné osy z a její směr nahoru odpovídá kladnému směru osy y.
- Funkce `LookAt` umožňuje nastavit pohled kamery podle zadané pozice a cílového bodu.
- Díky pohledové transformaci se můžeme ve scéně rozhlížet a procházet.

Pohledová transformace

Kameru popisujeme třemi údaji:

- pozicí kamery (eye),
- cílovým bodem, na který se kamera dívá (center),
- vektorem určujícím směr nahoru (up).



Pohledová transformace

Můžeme použít funkci **lookAt** z knihovny GLM.

- Eye – definuje bod, ve kterém bude kamera umístěna.
- Center – definuje bod, kterým se kamera dívá.
- Up – definuje, kterým směrem bude natočen 3D svět v kameře.

glm::mat4 view=glm::lookAt(eye, center, up);

$$\hat{v} = normalize(center - eye)\,,$$
$$\hat{r} = normalize(\vec{v} \times \vec{up})\,,$$
$$\hat{u} = normalize(\vec{r} \times \vec{v})\,.$$

$$view = A \cdot B\,, A = \begin{bmatrix} r_x & r_y & r_z & 0 \\ u_x & u_y & u_z & 0 \\ -v_x & -v_y & -v_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, B = \begin{bmatrix} 1 & 0 & 0 & -eye_x \\ 0 & 1 & 0 & -eye_y \\ 0 & 0 & 1 & -eye_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Pohledová transformace

Vypočtěte pohledovou matici, kde kamera je zadaná takto: $eye = (1, 0, 0)$, $center = (0, 0, 0)$ a $up = (0, 1, 0)$.

$$\hat{v} = \frac{center - eye}{\|center - eye\|} = (-1, 0, 0)$$

$$\hat{r} = \frac{v \times up}{\|v \times up\|} = (0, 0, -1)$$

$$\hat{u} = \frac{r \times v}{\|r \times v\|} = (0, 1, 0)$$

Pohledová transformace

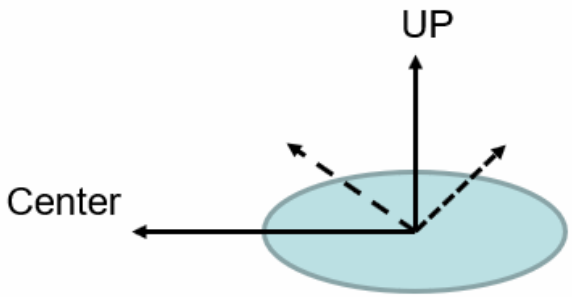
$$view = A \cdot B \, , A = \begin{bmatrix} r_x & r_y & r_z & 0 \\ u_x & u_y & u_z & 0 \\ -v_x & -v_y & -v_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} , B = \begin{bmatrix} 1 & 0 & 0 & -eye_x \\ 0 & 1 & 0 & -eye_y \\ 0 & 0 & 1 & -eye_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

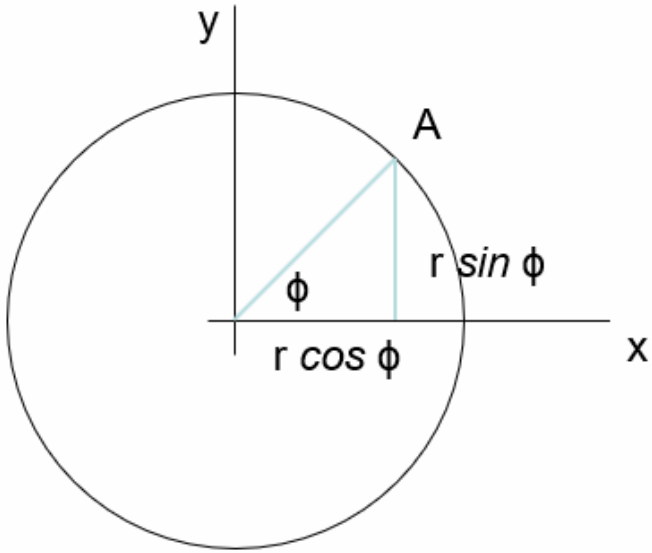
$$A = \begin{bmatrix} 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} , B = \begin{bmatrix} 1 & 0 & 0 & -1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$view = A \cdot B = \begin{bmatrix} 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & -1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Polární souřadnice vs. kartézské

Jak určit bod, na který se kamera dívá?



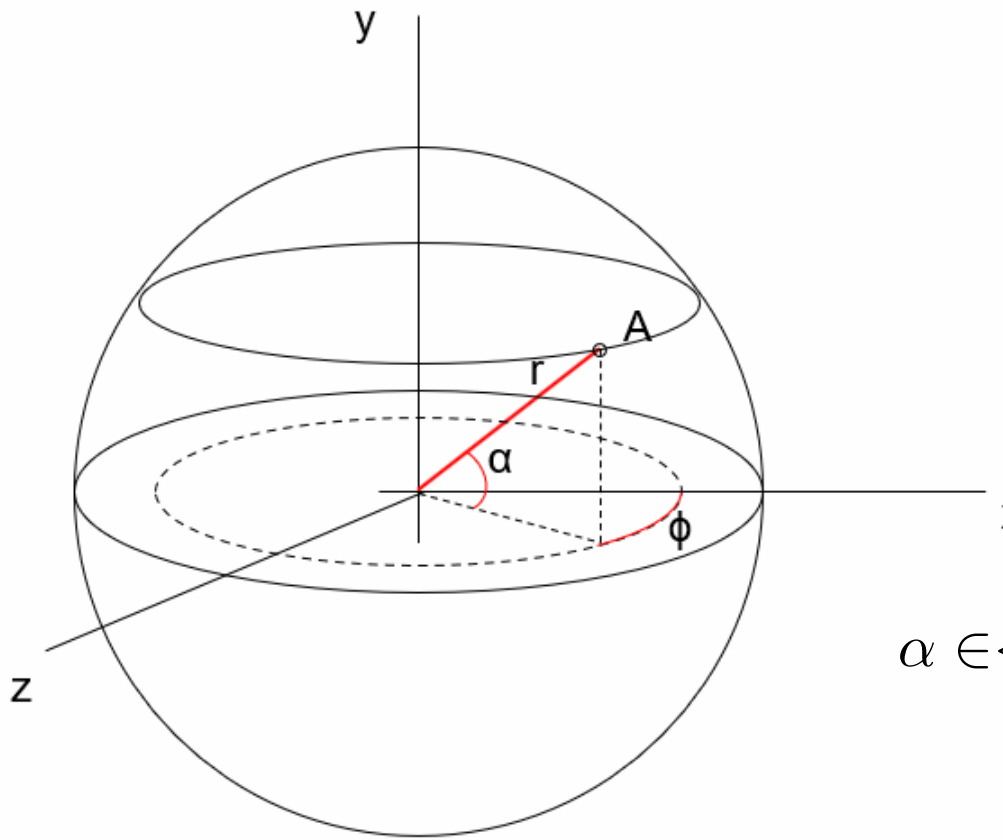


$$x = r \cdot \cos\phi$$

$$y = r \cdot \sin\phi$$

Sférický souřadný systém

Jak určit bod na který se kamera dívá ve 3D?



$$x = r \cdot \sin\alpha \cdot \cos\phi$$

$$y = r \cdot \cos\alpha$$

$$z = r \cdot \sin\alpha \cdot \sin\phi$$

$$\alpha \in < 0, 180 >, \phi \in < 0, 360 >$$

Základní pohyb kamery

Průchod scénou pomocí kláves (např. WSAD nebo šipky).

```
camera->toFront();
//eye+=(glm::normalize(target));

void Camera::toLeft(){
    eye+=(glm::normalize(glm::cross(target,up)));
}
```

Promítání

Promítání je převod z 3D do 2D.

- V geometrii nejprve volíme promítací metodu a potom v této zobrazujeme objekty. V počítačové grafice je to naopak.

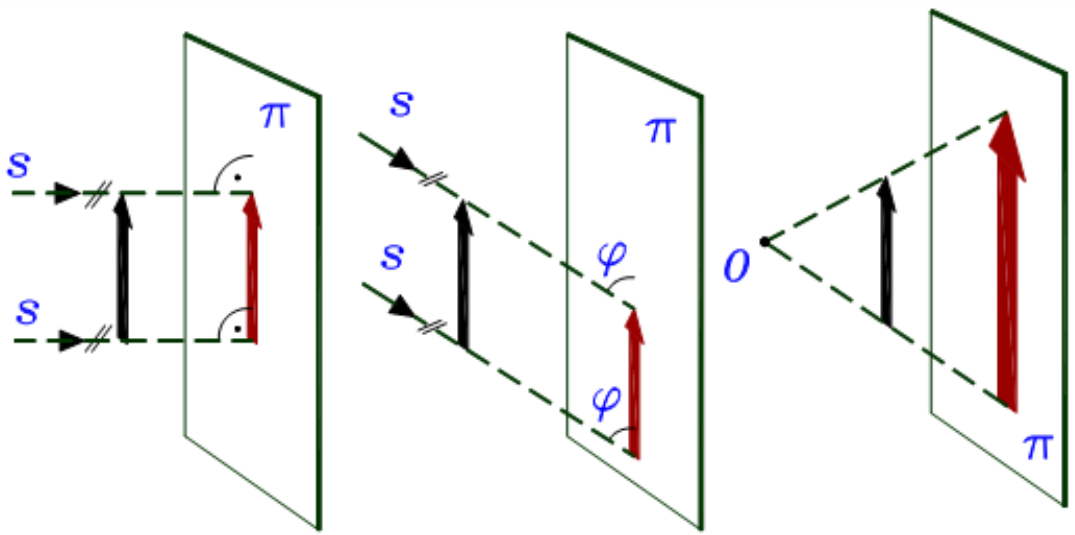
Čím je promítání definováno ?

- Promítací paprsky - polopřímka vycházející z promítacího bodu, směr závisí na typu promítání.
- Průmětna (viewing plane) - plocha v prostoru, na kterou dopadají promítací paprsky (paprsky vytvářejí průmět). Průmětnou nemusí být pouze rovina (polokoule, NURBS plocha apod.)

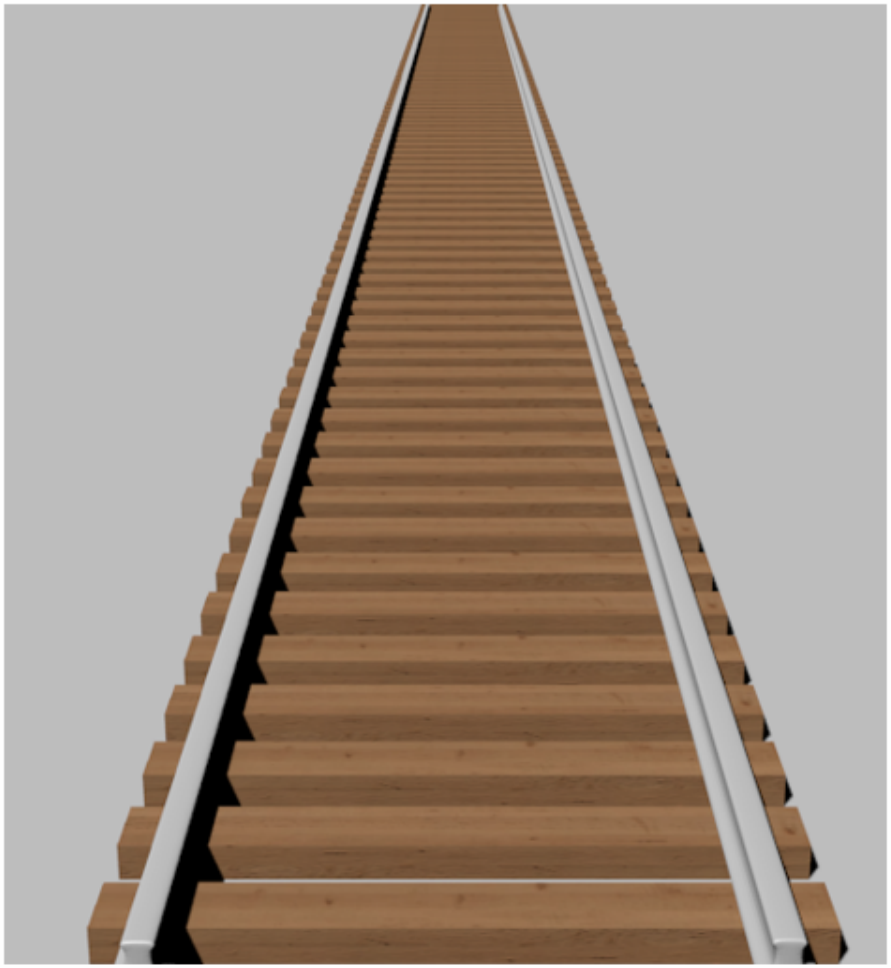
Klasifikace promítacích metod

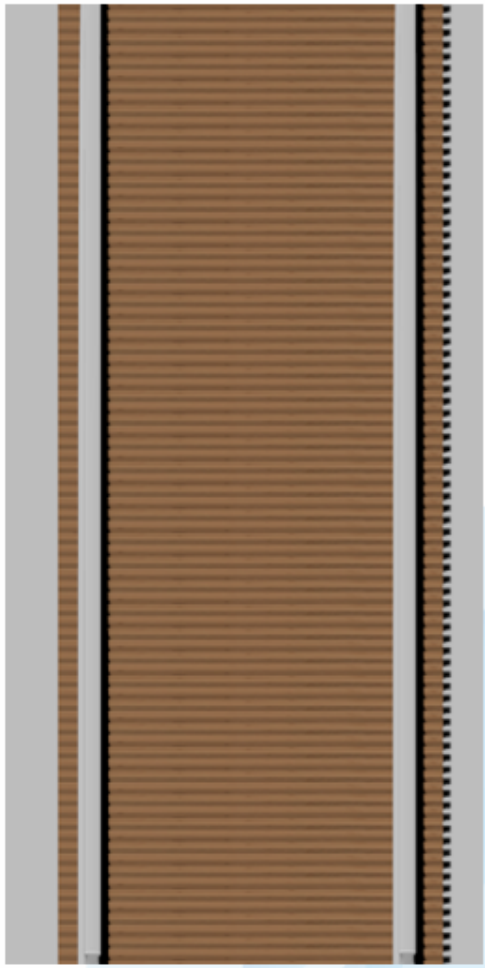
V počítačové grafice rozlišujeme dva základní typy promítání:

- rovnoběžné (paralelní, orthographic projections) - (pravoúhlé, kosoúhlé)
- středové (perspektivní, perspective projections)

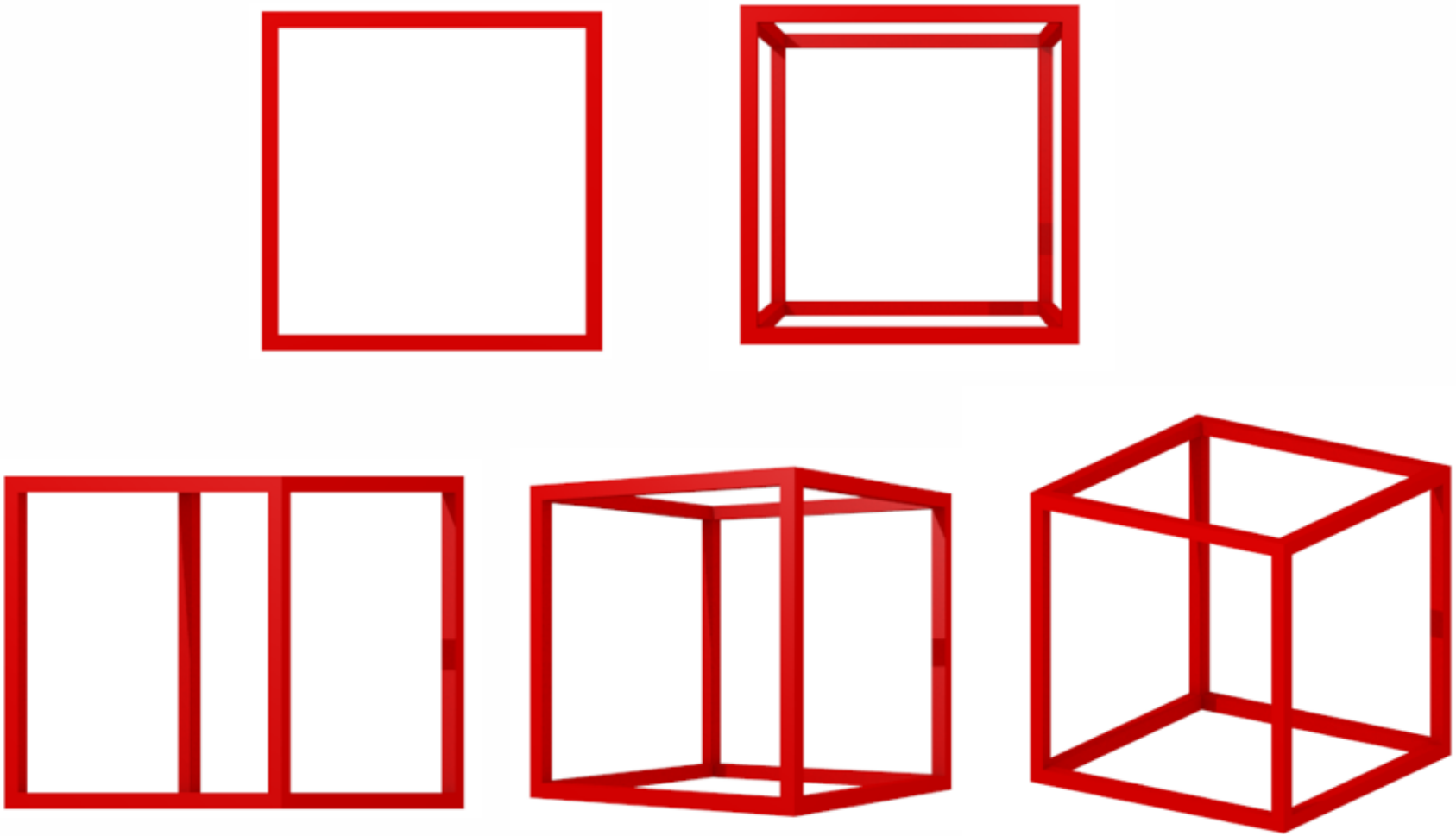


Projekce 3D do 2D



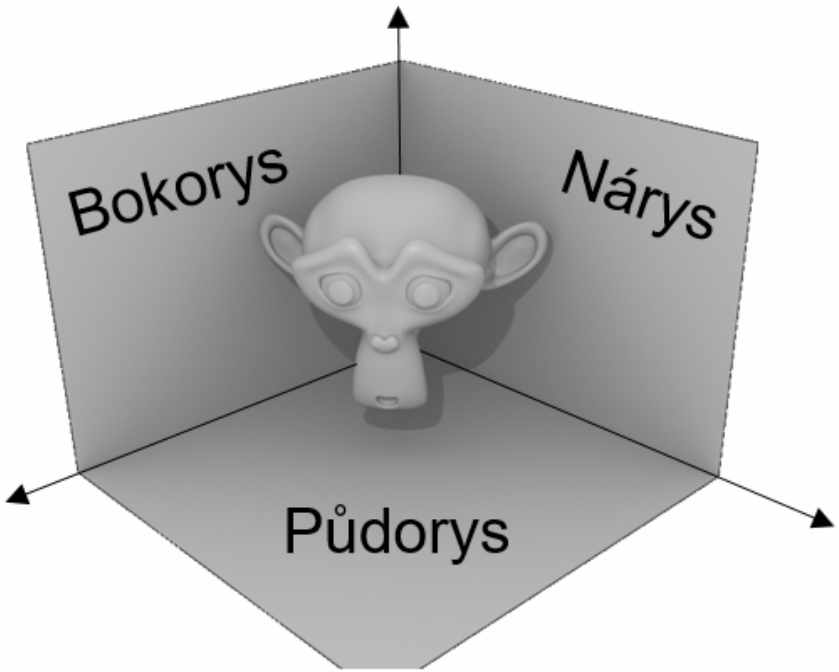


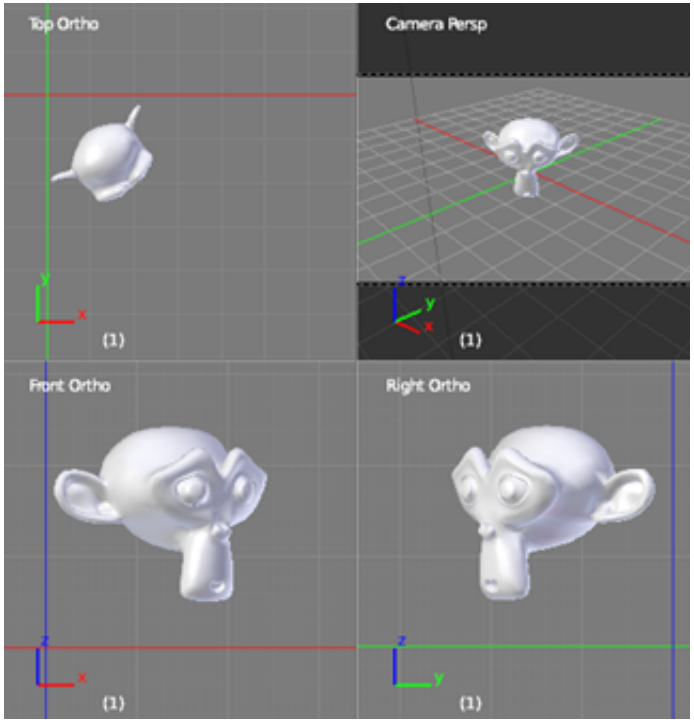
Promítání?



Promítání?

Rovnoběžné pravoúhlé promítání (nárys, půdorys a bokorys).

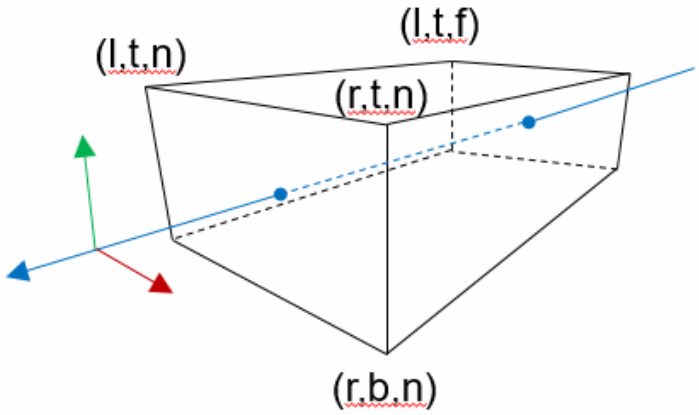


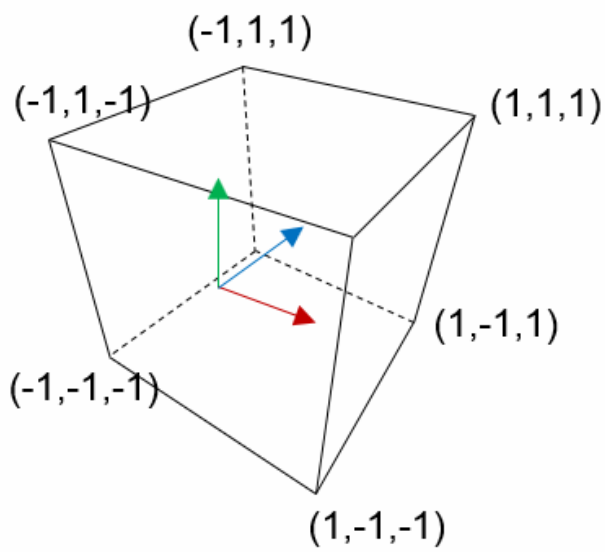


Ortogonální promítání

Scéna je reprezentována jako pravoúhlý hranol.

glm::ortho(left, right, bottom, top, zNear, zFar);
glm::mat4 projMat = glm::ortho(0, 400, 0, 400, -1, 400);



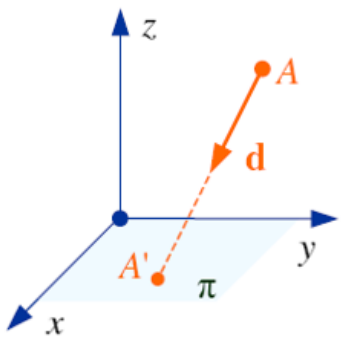


Ortogonální promítání

Matice popisuje rovnoběžné promítání na rovinu $z = 0$ (tedy na rovinu xy). Směr promítacího paprsku je $\vec{d} = (d_x, d_y, d_z)$

$$A = \begin{bmatrix} 1 & 0 & -\frac{d_x}{d_z} & 0 \\ 0 & 1 & -\frac{d_y}{d_z} & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

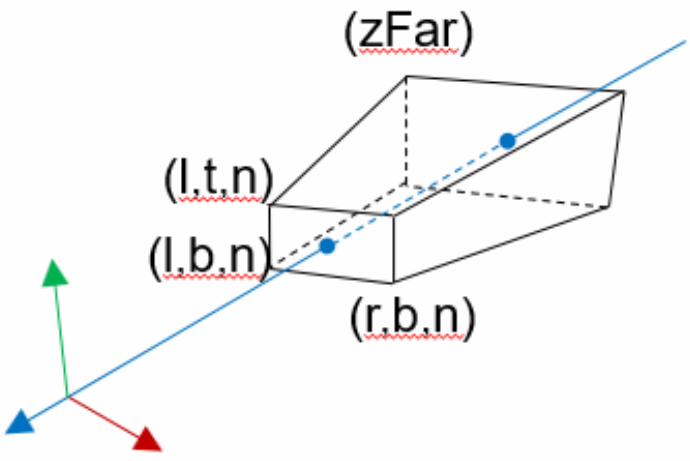
Rovnoběžným přímkám odpovídají opět rovnoběžné přímky, které však nemusí být rovnoběžné s původními přímkami.

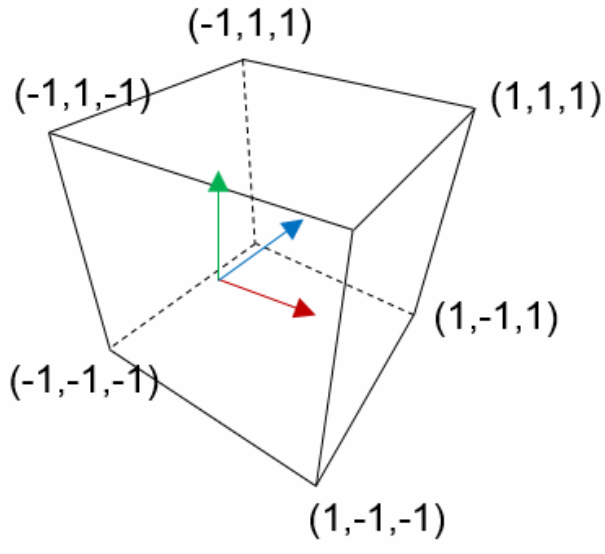


Perspektivní promítání

Scéna je definována jako komolý jehlan.

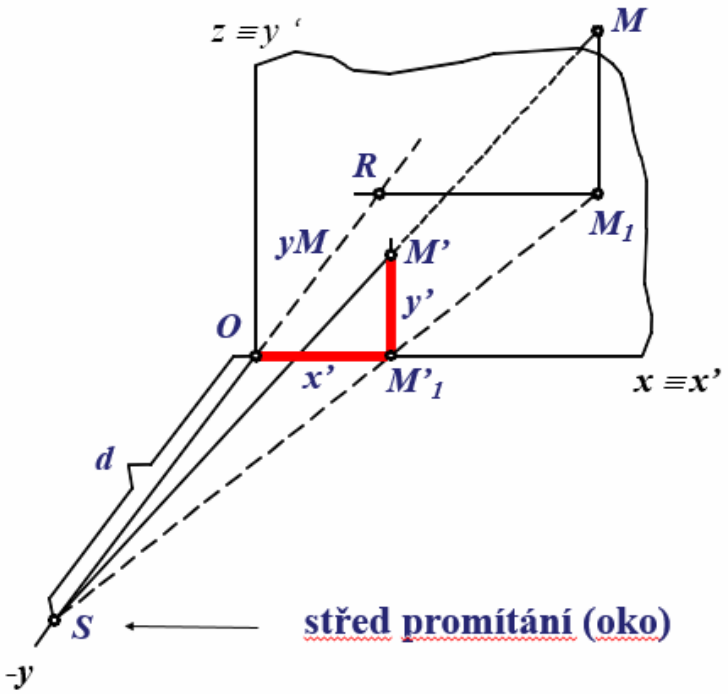
glm::frustum(left, right, bottom, top, zNear, zFar);
glm::perspective(fovy, aspect, zNear, zFar);
projection=perspective(45.0f, 4.0f/3.0f, 0.1f, 100.0f);
//radians or degrees





Perspektivní promítání

Vycházíme z podobnosti trojúhelníků.



$$\frac{x'}{x} = \frac{d}{d+y}$$
$$\frac{y'}{z} = \frac{d}{d+y}$$

$$x' = k \cdot x$$
$$y' = k \cdot z$$
$$k = \frac{d}{d+y}$$

Perspektivní promítání

Matice popisuje projekci ze středu o souřadnicích $(0, 0, f)$ na rovinu $z = 0$ (tedy na rovinu xy).

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -\frac{1}{f} & 1 \end{bmatrix}$$

glViewport

void glViewport(GLint x, GLint y, GLsizei width, GLsizei height);

- Část okna, ve které probíhá vykreslování.
- Transformace z NDC (normalizované souřadnice, $(-1, 1)$), na rozsah souřadnic obrazovky.
- Co vše se musí změnit pokud změníme velikost obrazovky?

```
void resizeWindow(int width, int height)
{
    glViewport(0, 0, width, height);
    aspectRatio = (float)width / (float)height;
    projection = glm::perspective(glm::radians(fov),
        aspectRatio, nearPlane, farPlane);
}
```

Křížový odkaz

```
//---- Camera.h ---
// dopredna deklarace
class Shader;
class Camera{
  private:
   Shader* sh;
  public:
   Camera(Shader* s);
   void metoda();
};

//--- Camera.cpp --
#include "Camera.h"
#include "Shader.h"
Camera::Camera(){

}
```

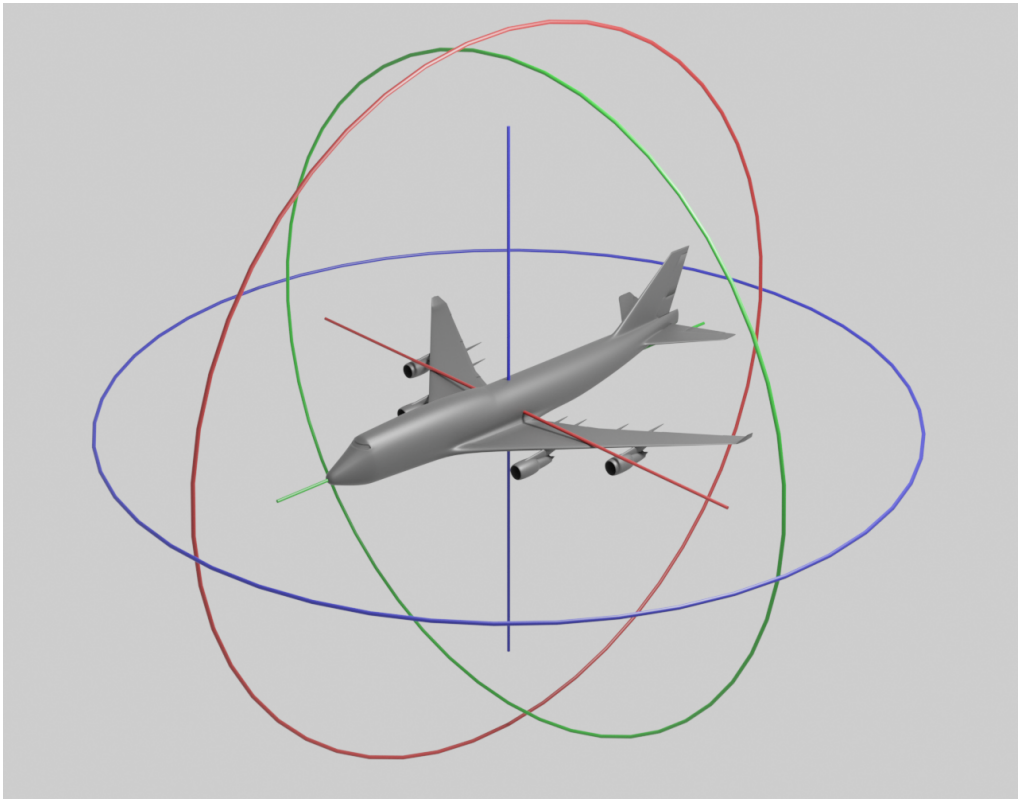
```
//------- Shader.h ------
// dopredna deklarace
class Camera;
class Shader{
  private:
     Camera* cam;
  public:
     Shader(Camera* c);
     void metoda();
};

//--- Shader.cpp ---
#include "Shader.h"
#include "Camera.h"
Shader::Shader(){

}
```

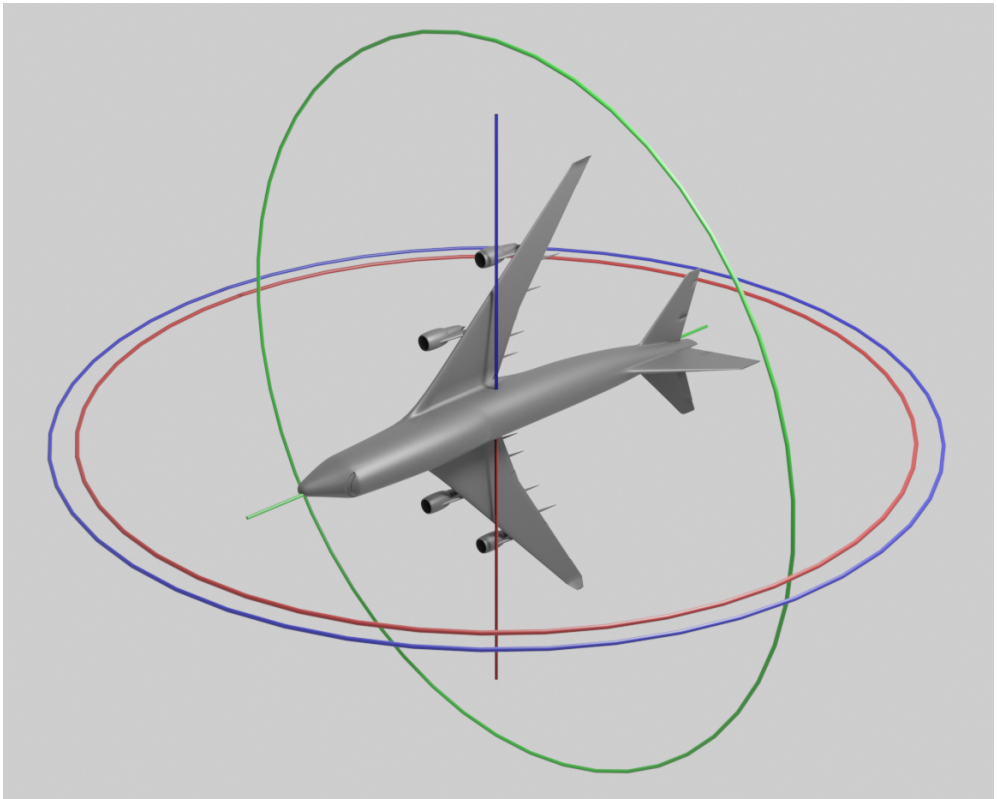
Gimbal lock

Gimbal lock je jev, který vzniká při použití Eulerových úhlů a skládání rotací (např. kolem os x, y a z). Pokud např. provedeme rotaci o 90° kolem osy y tak se osy x a z zarovnají, čímž vzniká **Gimbal lock** a ztrácí se jeden stupeň volnosti.



Gimbal lock

Natočením objektu kolem osy y o 90° došlo k zarovnání zbývajících dvou os rotace, což způsobí ztrátu jednoho stupně volnosti; tento problém lze řešit použitím kvaternionů.



Rotace

Matematický zápis složené rotace bodu A pomocí tří rotačních matic (Eulerovy úhly), kde rotace probíhá postupně kolem osy x o úhel α, poté kolem osy y o úhel β a nakonec kolem osy z o úhel γ.

$$A' = \mathbf{R}(\gamma, \beta, \alpha) \cdot A = \mathbf{R}_z(\gamma) \cdot \mathbf{R}_y(\beta) \cdot \mathbf{R}_x(\alpha) \cdot A$$

$$\begin{bmatrix} \cos(\gamma) & -\sin(\gamma) & 0 \\ \sin(\gamma) & \cos(\gamma) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos(\beta) & 0 & -\sin(\beta) \\ 0 & 1 & 0 \\ \sin(\beta) & 0 & \cos(\beta) \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\alpha) & -\sin(\alpha) \\ 0 & \sin(\alpha) & \cos(\alpha) \end{bmatrix}$$

```
mat4 R =
    rotate(mat4(1.0f), radians(gamma), vec3(0,0,1)) *
    rotate(mat4(1.0f), radians(beta),  vec3(0,1,0)) *
    rotate(mat4(1.0f), radians(alpha), vec3(1,0,0));
```

Kvaterniony

Kvaterniony (z lat. *quaternion*, čtveřice) jsou rozšířením komplexních čísel, které zavedl v roce 1843 irský matematik William Rowan Hamilton.

- Jsou tvořeny jednou reálnou a třemi imaginárními složkami.
- Na rozdíl od běžného násobení čísel není násobení kvaternionů komutativní.
- Umožňují efektivně reprezentovat a skládat rotace v prostoru.
- Rotaci lze popsat pomocí osy rotace a úhlu otočení.

Obecný kvaternion má tvar

$$q = a + bi + cj + dk = (a, \vec{v}),$$

kde $a \in \mathbb{R}$ je skalární část, $\vec{v} = (b, c, d)$ je vektorová část a platí

$$i^2 = j^2 = k^2 = ijk = -1.$$

Rotace v prostoru pomocí kvaternionů

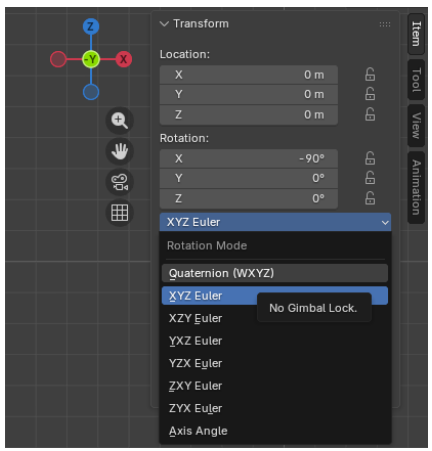
Rotaci o úhel θ kolem jednotkové osy $\vec{u} = (u_x, u_y, u_z)$ popisuje jednotkový kvaternion

$$\mathbf{q} = \left(\cos \frac{\theta}{2}, \vec{u} \sin \frac{\theta}{2} \right).$$

Rotaci bodu A provedeme pomocí vztahu

$$A' = \mathbf{q} A \mathbf{q}^{-1},$$

kde $\mathbf{q}^{-1}$ je inverzní kvaternion.
Některé nástroje umožňují výběr typu rotace.

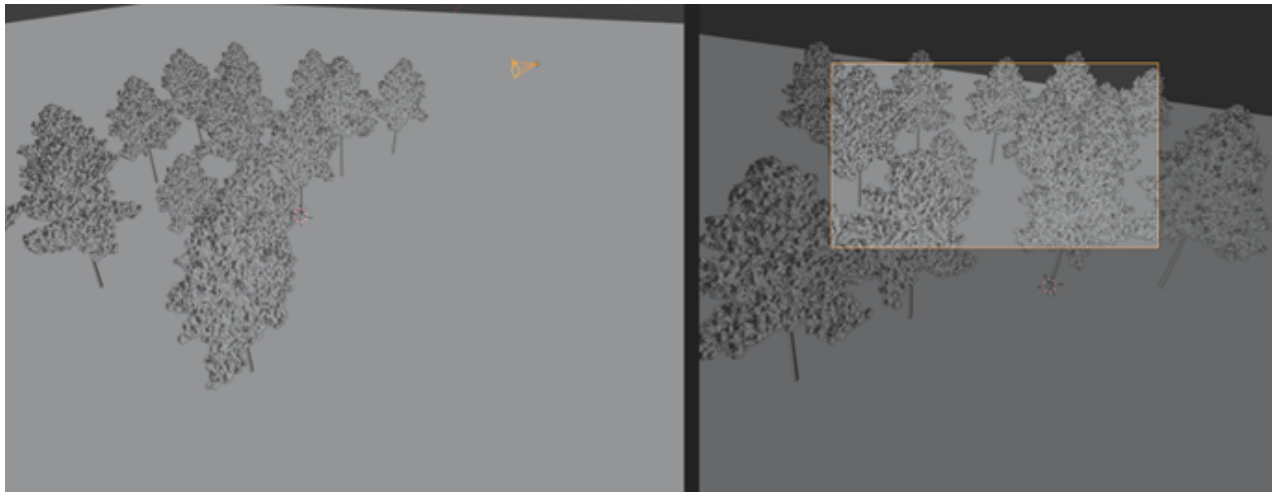


Cvičení – Model kamery

Navrhněte model kamery pro vykreslování 3D scény.

1. **Poloha a orientace kamery** - LookAt(eye, center, up);

2. **Typ a parametry projekce**
   - Perspektivní promítání – FOV, poměr stran, z-near a z-far.
   - Rovnoběžné (ortografické) promítání – hranice oblasti.



Kamera

```
#version 400
layout(location = 0) in vec3 vecPosition;
layout(location = 1) in vec3 vecNormal;
layout(location = 2) in vec3 vecColor;

out vec4 worldPos;

uniform mat4 modelMatrix = mat4(1.0);
//uniform mat4 viewMatrix;
//uniform mat4 projectionMatrix;

void main(void)
{
  worldPos = modelMatrix * vec4(vecPosition, 1.0);
  gl_Position = worldPos;
}
```

Draw Calls

Draw Call je příkaz, kterým CPU žádá GPU o vykreslení trojúhelníků tvořících 3D model (části scény).

Jak fungují?

- Každý draw call zahajuje požadavek na vykreslení pomocí příkazů, jako je `glDrawArrays()` nebo `glDrawElements()`.
- Počet draw callů závisí na rozdělení geometrie, materiálech, shaderech a způsobu vykreslování.
- Jeden model může vyžadovat více draw callů, například pokud používá několik materiálů.

```
// Draw a triangles
// mode,first,count
glDrawArrays(GL_TRIANGLES, 0, vertexCount);
```

Draw Calls

Proč jich mít méně?

- Každý draw call přináší režii na straně CPU.
- Příliš mnoho malých požadavků může omezovat výkon a způsobit, že GPU čeká na další příkazy.
- Seskupením geometrie se stejným materiálem lze často vykreslit více objektů najednou.

Příklad: 100 objektů vykreslovaných samostatně může vyžadovat 100 draw callů. Pokud lze jejich geometrii sloučit, může stačit jediný.

Vertex Colors

Vertex Color je barva přiřazená jednotlivým vrcholům (vertexům).

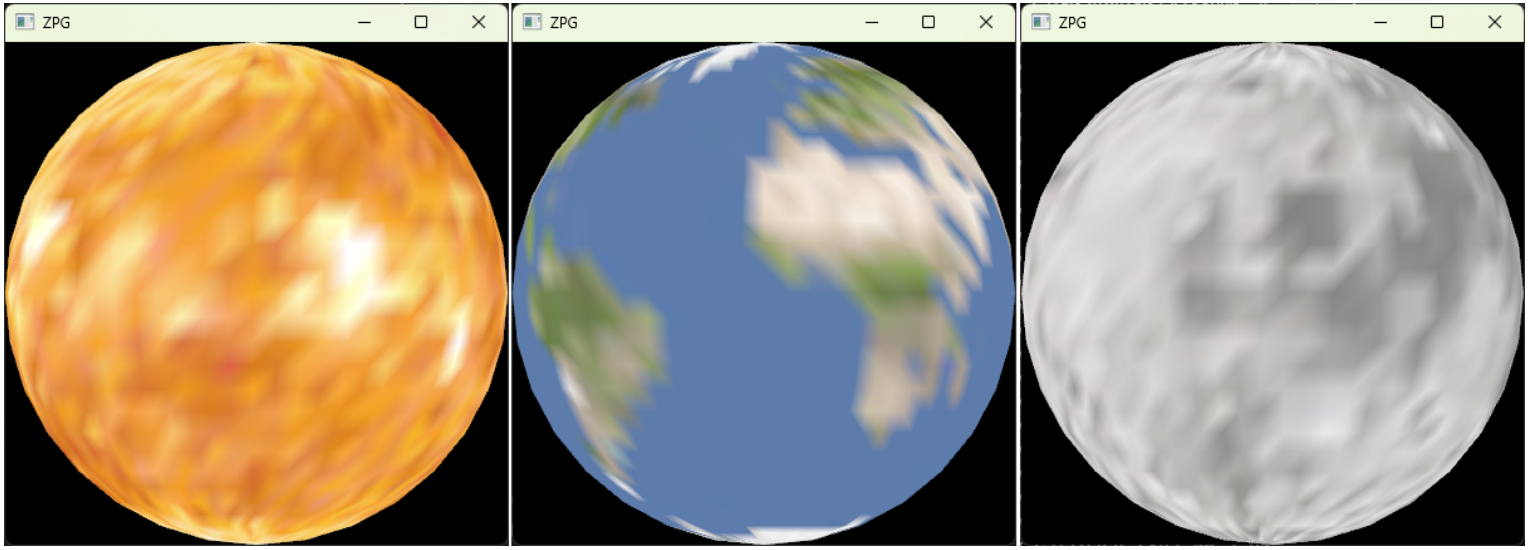
U modelu barvu obvykle obstarávají textury, ale v některých případech je vhodné uložit barvu přímo u jednotlivých vrcholů.

- obarvení objektu bez použití textury,
- předávání dodatečných dat do shaderu,
- ovlivnění výpočtu osvětlení nebo jiných efektů.

```
layout(location = 0) in vec3 position;
layout(location = 1) in vec3 color;
layout(location = 2) in vec3 normal;
```

Vertex Colors

Příklad použití vertex colors.



Vertex Colors

Na tomto cvičení použijeme následující model hodin, který se ale pokusíme vykreslit pouze jedním drawCallem. Využijeme předem připravené vertex colors pro jednotlivé části.

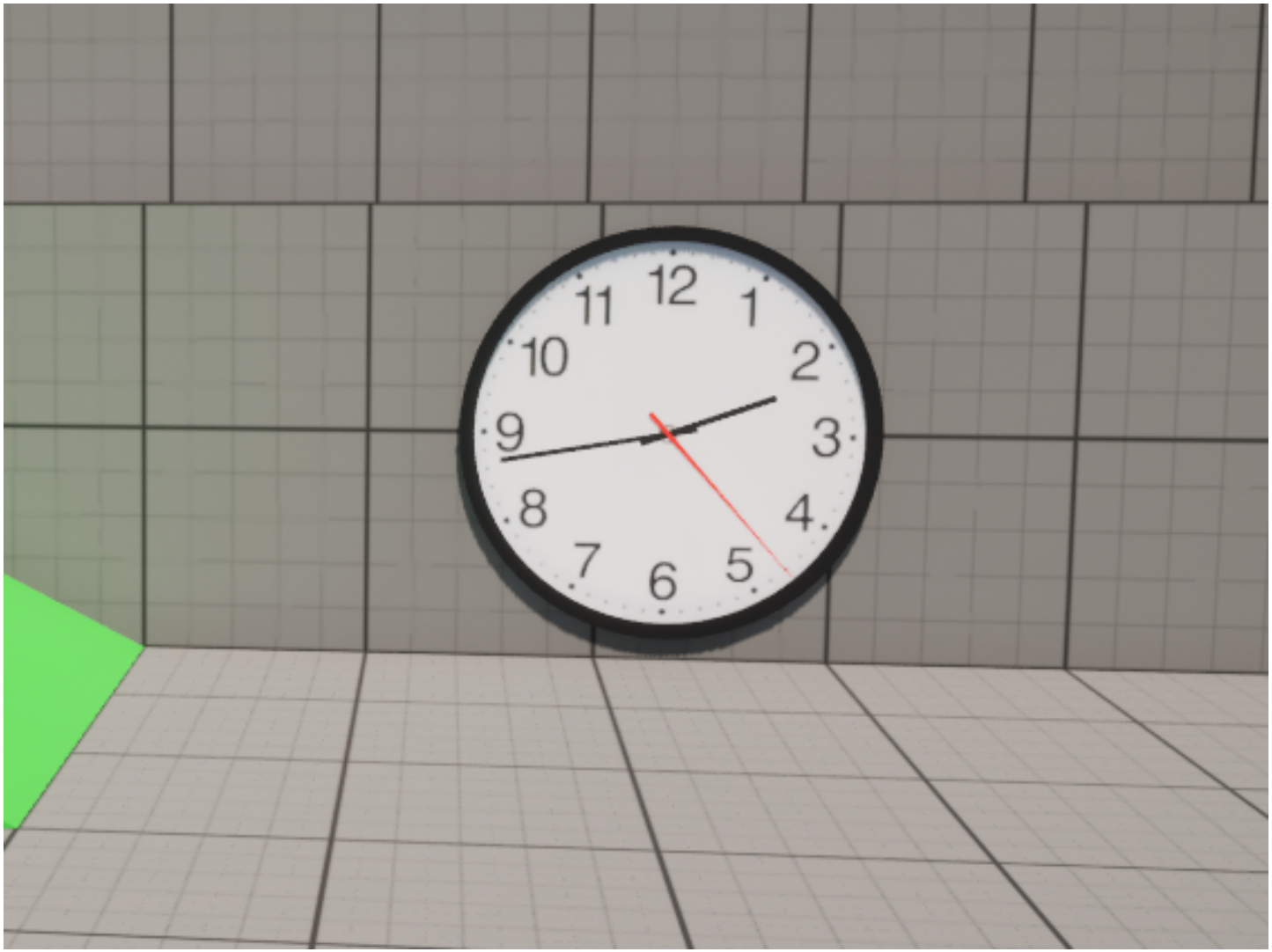


Vertex Colors

V modelu jsou použity základní barvy, kterými lze rozlišit, kam aktuální vrchol patří (pozadí, ručičky, čísla, logo apod.). Cílem není model pouze vykreslit, ale vykreslit správně s aktuálním časem a doplnit o kameru a promítání (hodiny mohou být na stěně apod.).



Unreal Engine



Závěr

Dotazy?